| EXP NO: 2 | Multi-Layer Perceptron (Mlp) For Classification |
|---|---|

## AIM:

To develop a Multi-Layer Perceptron (MLP) for a simple classification task on the Iris dataset, experiment with different numbers of hidden layers and activation functions, and evaluate the model's performance using accuracy and loss.

## ALGORITHM:

1. Load the Iris dataset and standardize the features using StandardScaler.
2. Split the data into 80% training and 20% test sets (stratified, random_state = 42).
3. Define a function create_mlp(hidden_layers, activation) that builds a Keras Sequential model with the given hidden Dense layers and a 3-unit softmax output layer, compiled with the Adam optimizer and sparse categorical cross-entropy loss.
4. For each hidden-layer configuration ([8], [16, 8], [32, 16, 8]) and each activation function (relu, tanh, sigmoid), train the model for 30 epochs with batch size 8 and 10% validation split.
5. Evaluate every model on the test set and record the test accuracy and loss; print the table of results.
6. Train the final model ([32, 16, 8], relu) and plot its training/validation accuracy and loss curves.
7. Read the four flower measurements from the user, scale them with the same scaler and predict the Iris species.

## CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
classes = iris.target_names

# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create MLP
def create_mlp(hidden_layers, activation):
    model = Sequential()
    model.add(
        Dense(
            hidden_layers[0],
```

```
            input_shape=(4,),
            activation=activation
        )
    )
    for units in hidden_layers[1:]:
        model.add(
            Dense(units, activation=activation)
        )
    model.add(
        Dense(3, activation="softmax")
    )
    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

# Configurations
hidden_layer_configs = [
    [8],
    [16, 8],
    [32, 16, 8]
]
activations = [
    "relu",
    "tanh",
    "sigmoid"
]
results = []

# Train models
for hidden_layers in hidden_layer_configs:
    for activation in activations:
        print("\n--------------------------------")
        print("Hidden Layers:", hidden_layers)
        print("Activation:", activation)
        print("--------------------------------")
        model = create_mlp(
            hidden_layers,
            activation
        )
        history = model.fit(
            X_train,
            y_train,
            epochs=30,
            batch_size=8,
            validation_split=0.1,
            verbose=0
        )
        loss, accuracy = model.evaluate(
            X_test,
            y_test,
            verbose=0
        )
        print("Test Accuracy:", round(accuracy * 100, 2), "%")
        print("Test Loss:", round(loss, 4))
        results.append(
            (
                hidden_layers,
                activation,
                accuracy,
                loss
```

```
            )
        )

# Display results
print("\n\nFINAL RESULTS")
print("--------------------------------")
for result in results:
    hidden_layers, activation, accuracy, loss = result
    print(
        "Layers:",
        hidden_layers,
        "| Activation:",
        activation,
        "| Accuracy:",
        round(accuracy * 100, 2),
        "%",
        "| Loss:",
        round(loss, 4)
    )

# Train final model for prediction
model = create_mlp(
    [32, 16, 8],
    "relu"
)
history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=8,
    validation_split=0.1,
    verbose=0
)

# Plot accuracy
plt.figure(figsize=(7, 5))
plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("MLP Accuracy")
plt.legend()
plt.grid()
plt.show()

# Plot loss
plt.figure(figsize=(7, 5))
plt.plot(
    history.history["loss"],
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
```

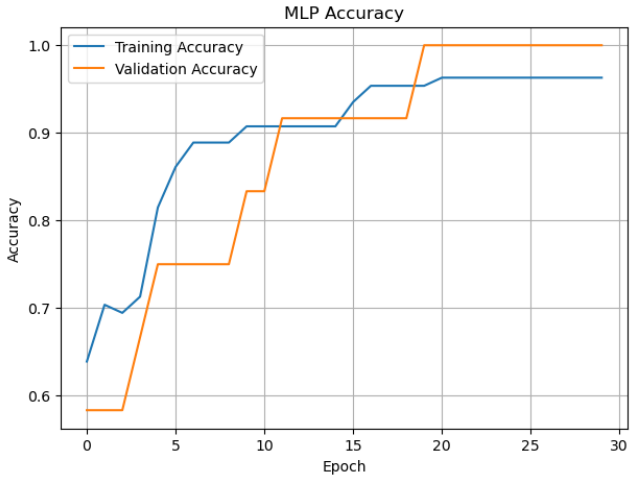
```
plt.title("MLP Loss")
plt.legend()
plt.grid()
plt.show()

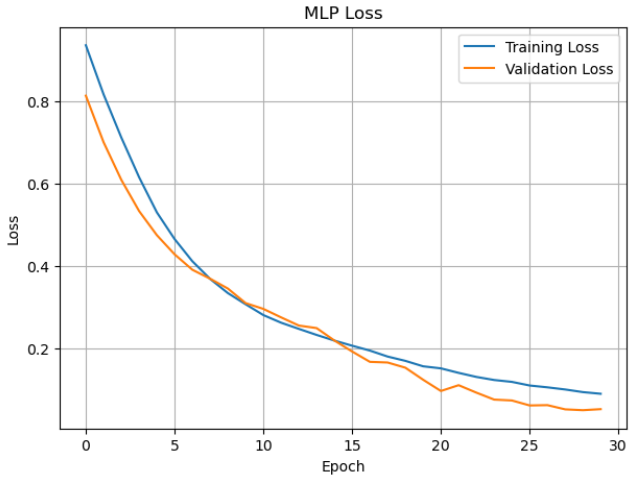
# User input prediction
print("\nEnter flower measurements")
sepal_length = float(input("Sepal Length (cm): "))
sepal_width = float(input("Sepal Width (cm): "))
petal_length = float(input("Petal Length (cm): "))
petal_width = float(input("Petal Width (cm): "))
user_input = np.array([
    [
        sepal_length,
        sepal_width,
        petal_length,
        petal_width
    ]
])
user_input_scaled = scaler.transform(user_input)
prediction = model.predict(
    user_input_scaled,
    verbose=0
)
predicted_index = np.argmax(prediction)
print("\nPredicted Iris Species:")
print(classes[predicted_index])
```

## OUTPUT:

```
--------------------------------
Hidden Layers: [8]
Activation: relu
--------------------------------
Test Accuracy: 76.67 %
Test Loss: 0.4694
--------------------------------
Hidden Layers: [8]
Activation: tanh
--------------------------------
Test Accuracy: 80.0 %
Test Loss: 0.4102
--------------------------------
Hidden Layers: [8]
Activation: sigmoid
--------------------------------
Test Accuracy: 80.0 %
Test Loss: 0.6485
--------------------------------
Hidden Layers: [16, 8]
Activation: relu
--------------------------------
Test Accuracy: 90.0 %
Test Loss: 0.2655
--------------------------------
Hidden Layers: [16, 8]
Activation: tanh
--------------------------------
Test Accuracy: 93.33 %
Test Loss: 0.2867
--------------------------------
Hidden Layers: [16, 8]
Activation: sigmoid
--------------------------------
Test Accuracy: 76.67 %
Test Loss: 0.7091
--------------------------------
```

```
Hidden Layers: [32, 16, 8]
Activation: relu
--------------------------------
Test Accuracy: 93.33 %
Test Loss: 0.1388
--------------------------------
Hidden Layers: [32, 16, 8]
Activation: tanh
--------------------------------
Test Accuracy: 96.67 %
Test Loss: 0.1555
--------------------------------
Hidden Layers: [32, 16, 8]
Activation: sigmoid
--------------------------------
Test Accuracy: 73.33 %
Test Loss: 0.7092
FINAL RESULTS
--------------------------------
Layers: [8] | Activation: relu | Accuracy: 76.67 % | Loss: 0.4694
Layers: [8] | Activation: tanh | Accuracy: 80.0 % | Loss: 0.4102
Layers: [8] | Activation: sigmoid | Accuracy: 80.0 % | Loss: 0.6485
Layers: [16, 8] | Activation: relu | Accuracy: 90.0 % | Loss: 0.2655
Layers: [16, 8] | Activation: tanh | Accuracy: 93.33 % | Loss: 0.2867
Layers: [16, 8] | Activation: sigmoid | Accuracy: 76.67 % | Loss: 0.7091
Layers: [32, 16, 8] | Activation: relu | Accuracy: 93.33 % | Loss: 0.1388
Layers: [32, 16, 8] | Activation: tanh | Accuracy: 96.67 % | Loss: 0.1555
Layers: [32, 16, 8] | Activation: sigmoid | Accuracy: 73.33 % | Loss: 0.7092
```




```
Enter flower measurements

Sepal Length (cm):  5
Sepal Width (cm):  2
Petal Length (cm):  8
Petal Width (cm):  3

Predicted Iris Species:
virginica
```

## RESULT:

The MLP was trained and evaluated for nine combinations of hidden layers and activation functions on the Iris dataset. The highest test accuracy, 96.67%, was obtained with the [32, 16, 8] network using tanh, and the lowest test loss, 0.1388, with [32, 16, 8] using ReLU. Sigmoid performed worst for the deeper networks (76.67% and 73.33%). For the entered measurements the final model predicted the species virginica.